CloudShield AI / DAIL

09 — Verification & Promotion Specification

Version 1.0 | Final Verification/Promotion Baseline | 1 September 2026

Purpose: Define the exact verification contracts, invariant predicates, evidence sufficiency rules, result semantics, failure handling, decision matrix, and transactional promotion rules that determine whether a Candidate State may become the next Trusted State. This document is the final safety gate between untrusted candidate analysis and trusted-state advancement.

Document Control

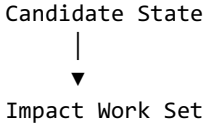
Field	Value
Document ID	CSAI-DAIL-VERIFY-009
Status	FINAL VERIFICATION/PROMOTION BASELINE
Parent documents	01 SRS; 02 Architecture; 03 Functional; 04 Terraform/AWS; 05 Domain/Data; 06 Lifecycle; 07 Identity/Dependency; 08 Impact/Invalidation
Primary inputs	CandidateState, TrustedState, ImpactReport, DependencyGraph, Verification Work Set
Primary outputs	VerificationReport + PromotionDecision + optional new TrustedState
Safety authority	Deterministic verification + Promotion Controller
LLM authority	None to self-certify or authorize promotion

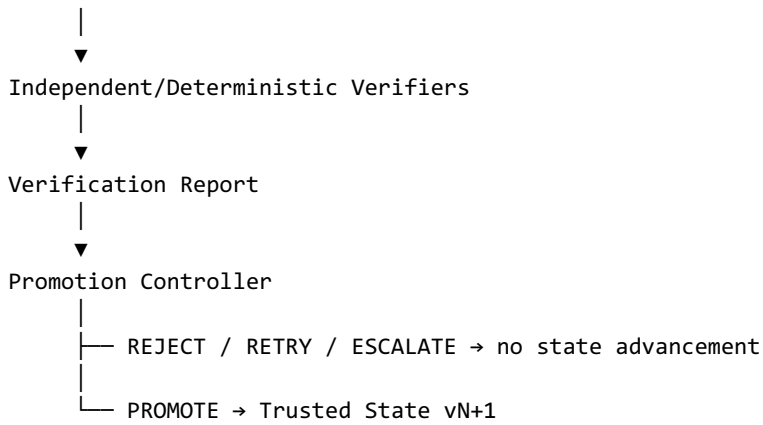
1. Purpose and Scope

This specification owns the final evidence-based decision process. It defines what must be verified, how results are interpreted, what evidence is sufficient, when uncertainty blocks promotion, and how a successful promotion atomically creates a new Trusted State.

- Verification request construction.
- Security and functional invariant verification.
- Structural verification.
- Evidence sufficiency.
- PASS/FAIL/UNKNOWN/UNSUPPORTED/VERIFIER_ERROR semantics.
- Promotion eligibility.
- REJECT/RETRY/ESCALATE decisions.
- Stale-parent protection.
- Transactional state advancement.
- Decision evidence.

2. Trust Boundary





The Promotion Controller may consume verification results but may not replace missing proof with intuition, LLM confidence, or a favorable heuristic.

3. Verification Principles

- Verify the exact candidate state being considered.
- Verify affected obligations identified by Impact Analysis.
- Run mandatory structural checks on every candidate.
- Use independent or deterministically derived evidence where practical.
- Treat UNKNOWN and UNSUPPORTED as lack of proof, not success.
- Treat verifier errors as non-success unless an explicit safe policy says otherwise.
- Keep evidence sufficient to reproduce and audit the decision.
- Promotion requires all blocking conditions to be satisfied.

4. Verification Result Semantics

Result	Meaning	Promotion implication
PASS	Verifier established that the target property holds for the candidate.	May satisfy the corresponding requirement.
FAIL	Verifier established that the target property does not hold.	Blocks promotion when target is required.
UNKNOWN	Verifier could not establish truth or falsity.	Does not satisfy required proof; normally blocks promotion.
UNSUPPORTED	Required property is outside the supported verifier/model.	Does not satisfy required proof; normally blocks promotion.
VERIFIER_ERROR	Verifier failed to execute or produced an invalid result.	Does not satisfy required proof; blocks promotion unless explicitly recovered.

5. Verification Classes

Class	Purpose
STRUCTURAL	Confirm candidate is syntactically/semantically valid within the supported Terraform/AWS model.
SECURITY	Evaluate protected security invariants.
FUNCTIONAL	Evaluate protected functional invariants.
BASELINE	Evaluate newly created resources against applicable

	safety predicates.
CONSISTENCY	Confirm internal state/graph/report consistency.

6. Mandatory Candidate Gate

Before invariant verification begins, the candidate must pass basic construction and consistency checks.

- Terraform/configuration parsing succeeds.
- Normalization succeeds under supported schema.
- No invalid resource identity structure.
- Candidate parent state exists.
- Canonical candidate hash is valid.
- Dependency graph construction succeeds or explicitly records supported uncertainty.
- Required invariant definitions are available.

A candidate failing mandatory structural gates cannot be promoted.

7. Verification Work Set

The Impact Engine produces the work set. The Verification Engine executes it.

```
VerificationWorkItem {
  target_id
  target_type
  invariant_id | null
  reason
  scope
  required_evidence
  priority
  verifier_id
}
```

- Every BLOCKING work item must complete with PASS before promotion.
- Non-blocking advisory work cannot override a failed blocking check.
- Unresolved required work is treated as non-success.

8. Security Invariant — Canonical MVP

The canonical security invariant protects against public SSH exposure in the supported security-group model.

```
INV-SEC-001
Predicate:
  No applicable security-group ingress rule permits
  TCP/22 from 0.0.0.0/0 or equivalent public IPv4 scope.

Scope:
  Supported security-group ingress rules relevant to the protected workload.

Required evidence:
  Canonical SG/rule representation
  + deterministic predicate evaluation
```

```
  + relevant resource identity/dependency context
```

The implementation must use the exact invariant registry definition selected for the experiment; this specification establishes the verification contract, not a broader AWS security policy.

9. Functional Invariant — Canonical MVP

The canonical functional invariant protects the intended EC2→RDS TCP/5432 connectivity path.

```
INV-FUNC-001
Predicate:
  The intended EC2 source can reach the intended RDS
  destination over TCP/5432 within the supported
  Terraform/AWS semantic model.

Required context:
  EC2 identity
  RDS identity
  subnet/network context
  routing context
  security-group rules
  destination port/protocol
```

A successful structural graph path is not automatically equivalent to runtime packet-level proof. The selected verifier must declare what level of evidence it establishes.

10. Verification Evidence Levels

Level	Meaning
L0	Configuration/schema evidence only.
L1	Deterministic semantic predicate evaluation from normalized infrastructure.
L2	Independent oracle evidence derived from a separate implementation/model.
L3	Runtime AWS verification where explicitly supported.

Promotion policy must specify the minimum acceptable level per invariant. Higher-level evidence may strengthen a decision but cannot contradict a deterministic failing predicate.

11. Evidence Sufficiency

Evidence is sufficient only if it establishes the target property for the exact candidate state and compatible invariant/verifier version.

```
sufficient(evidence, target):
  target matches
  AND candidate/state context matches
  AND invariant definition/version matches
  AND verifier/version is recorded
  AND required fields are present
  AND result is PASS
```

A PASS without adequate evidence metadata is not sufficient for promotion.

12. Verification Request Construction

```
for work_item in work_set:
    build VerificationRequest(
        candidate_id,
        parent_state_id,
        target,
        invariant_version,
        scope,
        verifier_version
    )
execute(request)
persist(result)
```

Requests must reference the candidate hash/state context so a result cannot accidentally be attached to a different candidate.

13. Security Verification Procedure

1. Load candidate normalized security-group/rule representation.
2. Identify rules relevant to the invariant scope.
3. Canonicalize protocol, port, direction, and source range.
4. Evaluate the public-SSH predicate.
5. Produce PASS if no forbidden applicable rule exists.
6. Produce FAIL if a forbidden applicable rule exists.
7. Produce UNKNOWN/UNSUPPORTED if the verifier cannot establish the required property.
8. Persist the exact rule evidence and verifier metadata.

14. Functional Verification Procedure

9. Resolve intended EC2 source and RDS destination identity.
10. Confirm relevant subnet/network context is known within the supported model.
11. Resolve security-group membership and relevant ingress/egress rules.
12. Resolve supported route relationships.
13. Evaluate TCP/5432 requirements.
14. Return PASS only when the selected verifier establishes the complete predicate at its declared evidence level.
15. Return UNKNOWN/UNSUPPORTED when required context cannot be established.
16. Persist the evaluated path and supporting evidence.

15. Structural Verification Procedure

- Validate normalized resources.
- Validate resource references.
- Validate dependency graph node/edge consistency.
- Validate invariant references.
- Validate candidate parent lineage.
- Validate canonical hashes.

- Reject malformed or internally inconsistent candidate states.

16. Independent Oracle Boundary

Where an independent oracle is available, its result is recorded separately from the primary DAIL verifier.

```
Candidate
  ├── DAIL Verification → result A
  └── Independent Oracle → result B
```

- Oracle implementation must be independently maintained from the primary decision implementation where the experiment requires independence.
- Oracle results support research comparison and confidence in verifier behavior.
- Oracle results do not bypass the Promotion Controller.

17. Conflicting Evidence

Situation	Required outcome
Primary verifier PASS; independent oracle PASS	Evidence agrees; candidate may satisfy that check.
Primary PASS; oracle FAIL	Conflict must be recorded; promotion cannot treat the check as unambiguously proven without explicit resolution policy.
Primary FAIL; oracle PASS	Primary failure remains blocking unless the verifier is explicitly superseded by validated policy.
Primary UNKNOWN; oracle PASS	May be resolved only if policy explicitly accepts oracle as sufficient evidence.
Any verifier ERROR	No silent conversion to PASS.

Conflicts are evidence requiring resolution, not opportunities to choose the favorable result.

18. Verification Result Aggregation

```
aggregate(results):
    if any required result == FAIL:
        return FAIL
    if any required result in {UNKNOWN, UNSUPPORTED, VERIFIER_ERROR}:
        return INSUFFICIENT_PROOF
    if all required results == PASS:
        return PASS
```

The exact promotion policy may distinguish RETRY from ESCALATE for different non-success classes, but required proof must never be silently satisfied.

19. Promotion Preconditions

Precondition	Required
Candidate is structurally valid	YES
Candidate parent is current Trusted State	YES
Required impacted invariants verified	YES
All required results PASS	YES
No blocking UNKNOWN/UNSUPPORTED/ERROR	YES

Required evidence sufficient	YES
Promotion policy version recorded	YES
Candidate hash/report consistency	YES

20. Promotion Decision Matrix

Condition	Decision
All required checks PASS + evidence sufficient + parent current	PROMOTE
Any required check FAIL	REJECT
Required check UNKNOWN/UNSUPPORTED and retry could resolve it	RETRY
Verifier/system failure cannot be safely retried automatically	ESCALATE
Stale parent	REJECT/REQUEUE; never promote
Candidate construction failure	REJECT
Evidence conflict unresolved	ESCALATE or REJECT according to configured policy; never PROMOTE automatically

21. PROMOTE

PROMOTE means the candidate has satisfied the defined verification policy and is authorized to become the next Trusted State.

```
PROMOTE(candidate C, current vN):
  verify all promotion preconditions
  create Trusted vN+1
  attach current invariant evidence
  persist PromotionDecision
  commit atomically
```

22. REJECT

REJECT means the candidate is not acceptable as the next trusted state. The current Trusted State remains authoritative.

- Persist decision and blocking evidence.
- Preserve candidate/verification records.
- Do not modify current Trusted State.
- Permit later attempts to start from the unchanged trusted baseline.

23. RETRY

RETRY means the candidate attempt did not establish proof but a bounded, meaningful re-attempt is appropriate.

- Record retry reason.
- Increment attempt number.
- Keep parent Trusted State unchanged.
- Apply retry limits/backoff policy outside the core semantic decision.
- Do not treat repeated UNKNOWN as eventual PASS.

24. ESCALATE

ESCALATE means the automated system cannot safely decide.

- Record unresolved conditions.
- Preserve evidence/conflicts/errors.
- Keep Trusted State unchanged.
- Require explicit external handling if the workflow supports it.

25. Stale Parent Protection

```
candidate.parent_state_id != trusted.current_state_id
    => STALE_PARENT
    => promotion forbidden
```

The candidate must be re-evaluated against the current trusted baseline if promotion is still desired.

26. Promotion Transaction

```
BEGIN TRANSACTION

1. Lock/read current Trusted State.
2. Verify candidate.parent_state_id == current state.
3. Revalidate promotion decision inputs.
4. Create new Trusted State vN+1.
5. Create state-specific invariant references.
6. Attach valid current evidence.
7. Persist committed PromotionDecision.
8. Emit STATE_PROMOTED evidence/event.

COMMIT
```

Any failure before commit must leave the previous Trusted State current.

27. New Trusted State Construction

```
TrustedState(vN+1) {
  parent_state_id = vN
  resources = candidate.resources
  state_hash = canonical_hash(candidate.semantic_state)
  invariant_refs = current valid evidence for vN+1
  commit_decision_id = decision_id
}
```

- Only evidence valid for vN+1 may be marked PROTECTED.
- Affected invariants must have fresh verification evidence unless explicit carry-forward policy allows otherwise.
- The candidate remains linked to the new state for audit.

28. Carry-Forward Verification

Unchanged, unaffected invariant evidence may be carried forward only if Document 06/08 compatibility rules are satisfied.

- Invariant definition/version unchanged.
- No direct impact.
- No relevant dependency impact.
- No identity uncertainty.
- Evidence/verifier compatibility holds.
- Promotion policy explicitly permits carry-forward.

Carry-forward is a policy-controlled optimization, not an assumption.

29. Blocking Conditions

Blocking condition	Effect
Required security invariant FAIL	Block promotion.
Required functional invariant FAIL	Block promotion.
Required structural check FAIL	Block promotion.
Required invariant UNKNOWN	Block promotion unless explicit policy provides accepted alternate proof.
Required invariant UNSUPPORTED	Block promotion when property is mandatory.
Verifier ERROR	Block promotion until resolved/retried.
Insufficient evidence metadata	Block promotion.
Stale parent	Block promotion.
Unresolved evidence conflict	Block automatic promotion.

30. Verification Freshness

A verification result is fresh only for the candidate/state context and compatible versions for which it was generated.

```
fresh(result, candidate):
  result.candidate_hash == candidate.state_hash
  AND invariant version matches
  AND verifier version/context compatible
  AND evidence not invalidated
```

A result generated for a different candidate must never be reused merely because the candidate looks similar.

31. Candidate Consistency Check

- Candidate resource fingerprints agree with candidate hash.
- Identity report references candidate resources that exist.
- Impact report references the same candidate.
- Verification results reference the same candidate.
- Decision references the same candidate and parent state.
- Graph version matches impact report.

32. Decision Evidence

```
PromotionDecision evidence must establish:
  candidate identity
  parent state
  required checks
  results
  evidence sufficiency
  policy version
  final decision
  commit/new state ID when promoted
```

A reviewer should be able to reconstruct why the candidate was promoted or rejected without inspecting hidden application memory.

33. Canonical Regression Test

```
Trusted v0:
  SEC = PROTECTED
  FUNC = PROTECTED

Candidate:
  removes public SSH
  but breaks EC2→RDS TCP/5432

Verification:
  SEC = PASS
  FUNC = FAIL

Aggregate:
  FAIL

Decision:
  REJECT

Trusted current state:
  remains v0
```

This is a mandatory test of the core DAIL safety claim: a repair cannot be accepted by improving one protected property while breaking another.

34. Canonical Safe Promotion

```
Trusted v0:
  SEC = PROTECTED
  FUNC = PROTECTED

Candidate:
  removes public SSH
  preserves EC2→RDS TCP/5432

Verification:
  SEC = PASS
```

```
  FUNC = PASS

Evidence:
  sufficient

Parent:
  current

Decision:
  PROMOTE

Result:
  Trusted v1
```

35. Canonical Unknown Scenario

```
Candidate:
  modifies relevant network structure

Functional verifier:
  UNKNOWN because required dependency context cannot be established

Decision:
  RETRY if resolution is feasible
  otherwise ESCALATE

Never:
  UNKNOWN → PROMOTE
```

36. Canonical Unsupported Scenario

```
Candidate:
  uses an unsupported relationship required to prove a protected invariant

Verification:
  UNSUPPORTED

Decision:
  REJECT/ESCALATE according to policy

Never:
  UNSUPPORTED → PASS by assumption
```

37. Verifier Failure Handling

- Capture exception/error metadata.
- Mark result VERIFIER_ERROR.
- Do not convert exceptions to FAIL unless the verifier explicitly defines that semantic.
- Do not convert errors to PASS.
- Retry only when the failure class is known to be transient/recoverable.

- Escalate when safe automatic recovery is unavailable.

38. Idempotency

Promotion operations must be idempotency-safe.

- A repeated request for the same committed decision must not create multiple trusted states.
- Decision/state identifiers must allow duplicate submission detection.
- A committed state must not be committed again as a separate version from the same decision.

39. Concurrency

MVP may use a single promotion worker, but the semantic contract must still protect against concurrent stale candidates.

- Check current parent at promotion time.
- Use a transaction/lock or equivalent atomic compare-and-swap.
- Reject stale candidate promotions.
- Never assume the state observed during analysis remains current at commit time.

40. Audit and Reproducibility

- Persist verifier IDs/versions.
- Persist invariant IDs/versions.
- Persist candidate/state hashes.
- Persist policy version.
- Persist evidence references.
- Persist decision timestamp.
- Persist parent/new-state relationship.
- Preserve failed decisions.

41. Test Matrix

Test	Expected result
Security PASS + Functional PASS	PROMOTE.
Security FAIL	REJECT.
Functional FAIL	REJECT.
Security UNKNOWN	No promotion.
Functional UNKNOWN	No promotion.
Unsupported mandatory verifier	No promotion.
Verifier exception	VERIFIER_ERROR; no promotion.
Evidence insufficient	No promotion.
Stale parent	No promotion.
Conflicting oracle evidence	No automatic promotion without resolution.
DB commit failure	Previous trusted state remains current.
Duplicate promotion request	Idempotent; no duplicate state.

42. Acceptance Criteria

- Every affected required invariant is explicitly verified or validly carried forward under policy.
- PASS is backed by sufficient state-specific evidence.
- FAIL blocks required promotion.
- UNKNOWN does not satisfy required proof.
- UNSUPPORTED does not satisfy mandatory proof.
- VERIFIER_ERROR does not satisfy proof.
- Structural checks are mandatory.
- Stale candidates cannot promote.
- Promotion creates a new Trusted State atomically.
- Rejected/retried/escalated candidates leave the current trusted baseline unchanged.
- Promotion decisions are auditable and reproducible.
- Independent oracle evidence cannot silently override the trust policy.

43. Non-Goals

- Generating patches.
- Choosing which candidate to ask an LLM to generate.
- Computing dependency closure.
- Defining every AWS service's verification semantics.
- Replacing invariant definitions with generic security heuristics.
- Treating probabilistic confidence as proof.

44. Final Verification/Promotion Directive

DAIL shall promote only when the candidate has passed every mandatory verification condition with sufficient evidence, is based on the current Trusted State, and satisfies the explicit promotion policy. Failure, uncertainty, unsupported semantics, verifier errors, evidence conflicts, and stale-parent conditions must never be silently interpreted as success.

Promotion is therefore an evidence-backed state transition, not a model-confidence decision. The Promotion Controller is the final authority for trusted-state advancement, and its authority is bounded by deterministic policy and auditable evidence.

Appendix A — Decision Algorithm

```
def decide(candidate, work_set, verification_report, trusted, policy):
    if candidate.parent_state_id != trusted.current_id:
        return REJECT("STALE_PARENT")

    if not structural_checks_pass(candidate):
        return REJECT("STRUCTURAL_FAILURE")

    for item in work_set.blocking:
        result = verification_report.for_item(item)
        if result == FAIL:
            return REJECT("REQUIRED_CHECK_FAILED")
        if result in {UNKNOWN, UNSUPPORTED, VERIFIER_ERROR}:
            return policy.non_success_decision(result)
```

```
    if not evidence_is_sufficient(verification_report, policy):
        return REJECT("INSUFFICIENT_EVIDENCE")

    return PROMOTE
```

Appendix B — Promotion Commit Algorithm

```
def commit_promotion(decision, candidate):
    begin_transaction()

    current = trusted_repo.get_current(candidate.lineage_id)
    assert current.state_id == candidate.parent_state_id

    validate_decision(decision, candidate, current)

    new_state = create_trusted_state(
        parent=current,
        candidate=candidate,
        evidence=valid_evidence_for_new_state()
    )

    trusted_repo.save_promoted(new_state)
    decision_repo.commit(decision, new_state.state_id)
    evidence_repo.append(STATE_PROMOTED(...))

    commit()
    return new_state
```

Appendix C — Next Specification

Document 10 — LLM Integration Specification defines how LLM-generated candidate patches are requested, constrained, parsed, validated, budgeted, retried, and isolated from the deterministic verification and promotion authority.